When the Window Cannot See the Question: Token-Denominated Observation Windows Are Not Language-Neutral

Anonymous authors
Paper under double-blind review

Abstract

Observation-window evictors such as SnapKV score the key–value cache using only the last w tokens of the prompt. We show that this integer is a language-dependent threshold. In the common passages + question + instruction layout, an instruction longer than w hides the question from the scorer: on Qwen3-4B the block of tokens after the question is 25 tokens in English but 107 in Bengali and 167 in Telugu, and the shipped default w=64 costs the latter two 16 and 19 points of accuracy while leaving English untouched. The failure is not linguistic. Padding an English instruction reproduces it, a JSON response schema reproduces it, and a tokenizer that shortens the Bengali instruction moves the threshold with it. Recovery needs question-sized slack: a window four tokens past the instruction recovers nothing. We therefore set $w = c + Q_{90}$ from the tokenizer, the chat template and a held-out question-length percentile. Preregistered and at matched retained cache, this closes the English, Bengali and Telugu gap ($+2/-2/0$ pp), closes the schema tails, and returns the shipped default where that default already suffices. Where it does not close, we show the residual is not a visibility failure.

1 Introduction

Serving a language model involves a surprising number of integers that count tokens: context limits, cache budgets, decode caps. Because tokenizers are not language-neutral (Petrov et al., 2023), each of those integers means something different in different languages, and the consequences have been documented for the budgets a user can see, such as cost, latency and usable context (Marchisio et al., 2024; Marie & Fujita, 2025). This paper is about a token-denominated integer that users cannot see. It sits inside the scoring rule of a widely used family of KV-cache eviction methods, and its default value decides whether the model is allowed to look at the question.

SnapKV (Li et al., 2024) and its descendants do not read the whole prompt when they decide what to evict. They score the cached keys against the last w tokens of the prompt — the observation window — and keep the highest-scoring entries. The kvpress implementation ships w=64; the opt-in RocketKV path in TensorRT-LLM ships w=32. In the retrieval-QA layout passages + question + instruction + chat suffix, that window is consumed from the right. Write c for the number of tokens standing after the question. If $c > w$, the window contains no token of the question at all, and the scorer decides what to keep without seeing what was asked (Figure 1).

On Qwen3-4B, the same one-line QA instruction occupies 25 tokens in English, 107 in Bengali and 167 in Telugu. At the shipped default, English prompts are safe while Bengali and Telugu prompts are blind, and they lose 16 and 19 points of accuracy. The mechanism, however, is not about those languages. Padding the English instruction until c passes w reproduces the cliff without leaving English; attaching a JSON response schema does the same; and a tokenizer that encodes the Bengali instruction in 26 tokens rather than 102

moves the cliff along with it. High-fertility tokenizers are the pathway that makes a fixed instruction overflow a fixed window, not a ranking of languages by difficulty.

Clearing the trailing block turns out to be necessary but far from sufficient. A window set four tokens past c recovers nothing at all, and one that leaves an English-sized slack of sixteen tokens recovers most of the loss but not the last few points. What the scorer needs is enough of the question, which suggests measuring the quantity directly. We define query visibility as the share of the question inside the window, $V = |W \cap Q|/|Q|$, computable from the tokenizer and the template with no forward pass; Luo et al. (2026) showed that question visibility changes method rankings and left exactly such a continuous measure open.

The measurement then implies a rule with no free parameters: set the window to the trailing block plus a question-sized slack, $\hat{w} = c + Q_{90}$, where Q_{90} is a percentile of question length estimated on held-out questions. Both terms are computed offline, no delimiter is required, and because SnapKV protects the window from within the retained budget, no extra cache is bought. Preregistered, the rule closes the English, Bengali and Telugu gap on Qwen3-4B ($+2/-2/0$ pp against uncompressed); closes the English JSON-schema tails that a mis-sized earlier version of the same rule had broken by 23 points; and returns exactly the shipped constant of 64 for a tokenizer and language where that constant is already adequate. It does not close everywhere. On Bengali it leaves 6 points at Gemma-3-4b-it and 8 points at Qwen3-8B, and in both cases we show that the residual is not a visibility failure.

Contributions. (i) We localise a language-dependent failure of KV eviction to the relation between the observation window and the trailing block, using experiments in which language is held fixed and c moves: an English instruction pad, an English response schema, and a change of tokenizer. (ii) We measure the dose-response, showing that the slack must be question-sized, and introduce query visibility V as an input-side, zero-inference measure of it. (iii) We propose and preregister AutoWindow, $\hat{w} = c + Q_{90}$, one integer per model, language and template, and report both where it closes and where it does not. (iv) We diagnose the residual in the two cases where it does not close, showing that the question is fully visible there and the remaining error is of a different kind.

Scope. We study a method family rather than a shipped production default: vLLM exposes no SnapKV-style evictor, the RocketKV path is opt-in, and Mao et al. (2026) argue that research evictors rarely ship as defaults. We do not claim that KV-cache compression harms low-resource languages in general. We claim something narrower and more actionable: a token-denominated window blinds whichever prompts carry a trailing block longer than itself, and under today's tokenizers those prompts are systematically the non-English ones.

2 Related work

Observation windows and query visibility. Prefill eviction scores the cache from a last-w slice of the prompt (Li et al., 2024). Luo et al. (2026) showed that whether the question falls inside that slice changes the ranking of compression methods on English long-context benchmarks — the query-aware versus query-agnostic gap is largest for SnapKV among the methods they audit — and left a continuous measure of query sensitivity open, noting that the natural candidates require a forward pass. V fills that slot from the other side: it is a property of the scorer's input rather than of its output. SnapKV's accumulated attention and the estimated attention of Devoto et al. (2025) are computed from the model; V is computed from the tokenizer and the template. Expected Attention is the informative contrast, because it removes the need to see the question by estimating the distribution of future queries, and never reasons about an observation window at all.

How w gets chosen. Recent eviction work treats w as a hyperparameter to set rather than as a threshold to test. IntentKV (Zhao et al., 2026) and the original SnapKV retrieval configuration use w=64; Nawrot et al. (2026) sweep window sizes up to a few hundred tokens and select a task-average optimum. None of this work lets w fall below a trailing instruction whose length varies across otherwise matched items, which is the cell this paper occupies.

Instruction order, protection, and delimiters. Chen et al. (2026) document that eviction can discard an instruction in multi-instruction English prompts and propose a whitelist and a fair-eviction split; their reordering moves two instructions inside a system prompt, whereas we move the instruction relative to the question and vary its token length until it overflows the window, and our remedy requires no knowledge of prompt structure. Garcia (2026) show that protecting a prompt's prefix and suffix outperforms scoring under a global cache cap, assuming that the structurally critical tokens occupy a roughly fixed budget, on the order of 15 to 30 tokens for the Qwen chat template. On Qwen3-4B the Bengali and Telugu QA instructions occupy 102 and 162 tokens; we treat that assumption as a prediction and test it. FINCH (Corallo & Papotti, 2024) sizes its window from a user-supplied delimiter in a context–delimiter–question layout, so $V{=}1$ holds there by construction. We read that as evidence for the problem rather than against the remedy: it shows that w must adapt, but it adapts using a delimiter and a question-last assumption that the failure studied here does not enjoy.

Tokenizer fertility. Petrov et al. (2023) established that a fixed context budget is not language-neutral, and subsequent work has studied the cost, latency and usable context that follow, including under weight quantization (Marchisio et al., 2024; Marie & Fujita, 2025). Those are budgets the user can see and, in principle, plan around. The constant studied here sits inside the scorer, where it is invisible from the outside; to our knowledge no published eviction study attributes per-language damage to it.

3 Setup

Task and data. Every item is a question, its gold passage, and same-language distractor passages packed to a fixed 8k-token prefill, with the gold passage rotating through front, middle and back positions by item index. Questions and passages come from XQuAD (Artetxe et al., 2020) for English, Chinese, Spanish, Vietnamese and Thai, and from TyDiQA-GoldP (Clark et al., 2020) for Swahili, Bengali and Telugu. The two collections are not comparable to each other, and XQuAD is parallel only within itself, so we never compare accuracy across languages. Every number in this paper is a within-language, item-paired difference between two configurations evaluated on the same 100 items.

Models and compression. Qwen3-4B (Qwen Team, 2025) is the primary model; Qwen3-8B is a scale slice and Gemma-3-4b-it (Gemma Team, Google DeepMind, 2025) a tokenizer contrast. We compress with SnapKV (Li et al., 2024) through the kvpress implementation at a fixed eviction ratio of 0.75 — three quarters of the prefill cache is discarded — and expose the observation window as the treatment variable. SnapKV keeps the window inside the retained budget, so changing w at a fixed ratio does not change how much cache survives; every comparison below is at matched retained KV (Appendix A states the scoring step precisely, and Appendix L the production scope). Decoding is greedy with a 384-token cap, after we found that the more common 128-token cap is itself a token-denominated constant: it truncates answers in high-fertility scripts and inflates apparent damage.

Scoring. An answer counts as correct if a gold span appears, after Unicode normalization and language-aware tokenization, either inside the marked answer span or inside the first sentence of the prose. No LLM judge is used anywhere in this paper. The rule was fixed before the runs reported here and is applied offline to raw generations; a stricter, marker-only variant leaves every conclusion in the paper unchanged (Appendix C). Paired comparisons are two-sided McNemar tests on 100 items, and we report the discordant counts (fixed/broken) so the reader can see what the p-value is made of.

The quantities. Prompts are instruction-last unless stated: passages + question + instruction + chat suffix. Write I for the token length of the trailing instruction, s for the chat-template suffix that follows it, and $c{=}I{+}s$ for the number of tokens after the question. Write Q_i for the set of token positions holding item i's question, so that $|Q_i|$ is its length, and W for the set of positions in the last-w window. Query visibility is the share of the

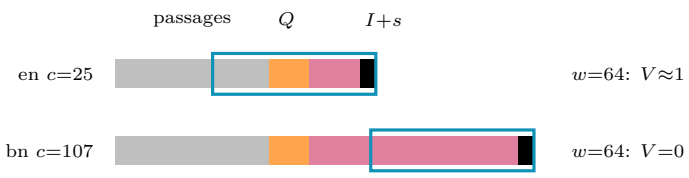

Figure 1: The instruction-last layout, drawn schematically (widths are not to token scale). The cyan bracket is the last-w observation window at the kvpress default w=64. With an English trailing block (c=25) the window still covers the question; with the Bengali one (c=107) it lies entirely inside the instruction, and the scorer ranks the passages without ever seeing what was asked.

question the scorer can see,

$$V_i \;=\; \frac{|W \cap Q_i|}{|Q_i|} \;=\; \mathrm{clamp}\left(\frac{w-c}{|Q_i|},\, 0,\, 1\right) \quad \text{in this layout,}$$

where we take the intersection as the definition and the closed form as a corollary, so that interleaved layouts do not silently break it. On Qwen3-4B the eight instructions tokenize to I between 20 (English) and 162 (Telugu) tokens; with s=5 this gives c from 25 to 167 (Appendix D). Both c and Q are produced by script from the tokenizer and template actually used in the run, never copied from a table; Appendix B gives the measurement procedure and Appendix I the frozen instructions and the exact prompt layout.

The held-out percentile. Q_{90} is the 90th percentile of question token length on a split that excludes the 100 evaluation items: XQuAD validation after index 100, and TyDiQA-GoldP train with any evaluation question removed (n=1090/2387/5563 for English, Bengali and Telugu). On Qwen3-4B this gives Q_{90}=18/76/80. It is never estimated on scored items and never tuned against accuracy.

Software stacks. Each comparison stays inside one model and one software stack (driver, CUDA, torch, kvpress); cells from different stacks, models or tokenizers are never pooled, even when they carry the same configuration name. The follow-up runs reported in Sections 5 and 6 re-ran cells that earlier stores already contained; across those 1,200 shared generations the new text is byte-identical to the old (Appendix J), which is what licenses reading the earlier and later tables as describing one system.

4 The window is a threshold

The window does not know that it is looking at an instruction. It scores whatever occupies the tail of the prompt (Figure 1). This section establishes three facts about that tail: damage tracks the window against the tail rather than against the language; it appears in English as soon as we lengthen the tail; and it appears at a constant that is already shipped.

A window sweep across five languages. Table 1 reports paired damage against the uncompressed baseline for five languages at five windows, at a fixed eviction ratio of 0.75. The predictions were registered before the sweep: blind when $w < c$, safe when $w > c$, with an allowed refinement that the step sits somewhat above c rather than exactly at it.

English never moves: its trailing block is 25 tokens, so every tested window contains the whole question, and the five cells run from +2 to −1 pp. Bengali is damaged in every blind cell (−13 to −21 pp, all $p < .01$) and comes back to −3 pp only at w=176, which is c+69. Telugu is damaged in all five cells, including w=176 — for Telugu that window is c+9 — where it is still −13 pp (p=.019). Thai and Swahili, whose trailing blocks are 45 and 47 tokens, dip at the one tested window below their c and are within noise elsewhere; we do not headline Swahili, whose baseline is 56%.

Two features of the table matter more than the average effect. First, the ordering of damage follows the ordering of c and not the ordering of baseline accuracy: Swahili has the weakest

Table 1: Sweeping the observation window. Paired Δpp against the uncompressed baseline, Qwen3-4B, n=100 items per cell, eviction ratio 0.75. Cells left of a language's c were predicted blind. Star: McNemar $p < .05$.

	c	base	w=32	56	88	120	176
en	25	93	+2	+1	+1	−1	+1
th	45	85	−6	−4	0	−2	−3
sw	47	56	−7*	−4	−2	−4	−3
bn	107	73	−13*	−15*	−21*	−8	−3
te	167	56	−19*	−21*	−18*	−14*	−13*

baseline of the five and close to the smallest loss, while Bengali and Telugu have the longest trailing blocks and the largest losses. Second, clearing c is evidently not sufficient — Telugu at c+9 is still down 13 points — which is the question Section 5 takes up.

The same cliff, constructed in English. If the mechanism is trailing length rather than language, then padding an English instruction should reproduce the cliff, and widening w past the new c should remove it. We pad the English QA instruction with content-free filler to four lengths, which put c at 57, 73, 105 and 137 tokens, and sweep five windows on the same English items (Figure 2b). Accuracy at w=32 is 81/86/82/78% against per-condition maxima of 92/94/93/91%.

The threshold moves with the pad. In each of the four conditions accuracy peaks at the first tested window that exceeds that condition's c, and at no earlier window: at w=80 for c=57 and c=73, and only at w=144 for c=105 and c=137. The sharpest case is the 96-token pad, where w=104 — one token short of its c=105 — scores 90% and w=144 scores 93%. Nothing about the language, the items, or the retained budget changed across these curves; only the number of tokens standing between the question and the end of the prompt did. In a separate matched run that carries its own uncompressed baselines, a 64-token pad costs 8 pp at the default w=64 (2 fixed / 10 broken, p=.039) while the unpadded English prompt costs nothing.

A constant that is already shipped. kvpress ships w=64; the opt-in RocketKV path in TensorRT-LLM ships w=32. Moving from the first constant to the second, on matched items, costs Thai 4.5 pp (2/11, p=.022) and Swahili 6.5 pp (1/14, p=.001). Both languages sit safely above the threshold at w=64 (c=45 and 47) and below it at w=32. The failure is therefore not confined to the two scripts with the longest instructions: it reaches any language whose trailing block outgrows whichever constant is in force. A third handle points the same way. Moving the instruction to the front of the prompt puts the question inside every window without touching the press or the retained budget, and recovers Bengali by 14 pp (Appendix K); we use it as a lever on the mechanism, not as a deployment recommendation, since a chat deployment that ends with the question already has V=1.

5 The slack must be question-sized

Section 4 leaves one quantity unmeasured. If the window has to reach past the trailing block before the scorer sees the question, how far past? Telugu at c+9 was still 13 points down, so "clear c" is not the answer.

A dose ladder. We sweep five windows at fixed offsets above the measured c on Telugu, the language with the longest trailing block and the deepest hole: c+4, c+16, c+32, c+48 and c+80, on 100 items with their own uncompressed baseline (Table 2, Figure 2c). The offsets were fixed before the run and c was re-measured on the run tokenizer.

Four visible tokens of question buy nothing: c+4 loses 18 pp, which is what the same items lose at the shipped w=64, where they see none of the question at all. The curve then rises with the slack and reaches the baseline only at c+80. A partial Bengali replication reproduces the first two rungs — c+4 costs 17 pp (3/20, p<.001) and c+16 costs 7 pp —

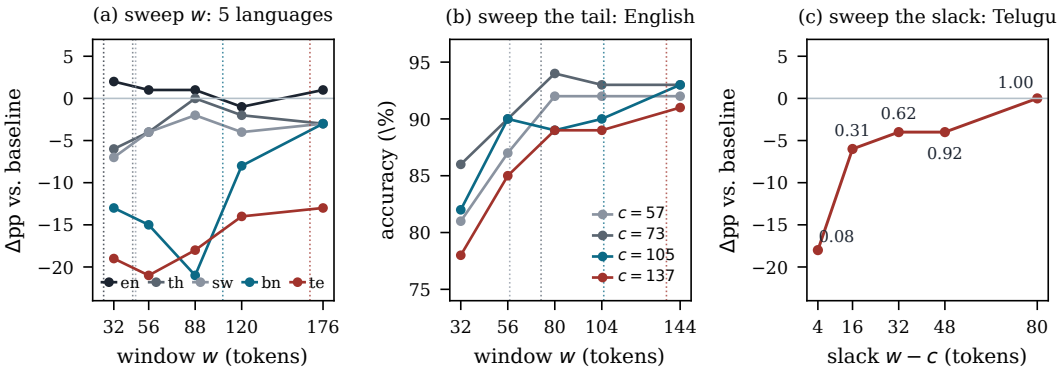

Figure 2: Three sweeps of the same mechanism, each paired against its own uncompressed baseline and scored by the same rule. (a) Five languages against the window; dotted verticals mark each language's c. (b) English only, four instruction pads; dotted verticals mark each pad's c, and accuracy peaks at the first window that clears it. (c) Telugu, five windows at fixed offsets above c; point labels are the median per-item visibility V. Panels (a) and (c) share their vertical scale.

Table 2: The dose ladder. Telugu, c=167, uncompressed baseline 56%, n=100, eviction ratio 0.75, paired McNemar. Median V is the median per-item share of the question that falls inside the window.

w	$w-c$	median V	accuracy	Δpp	fixed/broken	p
171	4	0.08	38	−18	5/23	.0009
183	16	0.31	50	−6	5/11	.21
199	32	0.62	52	−4	4/8	.39
215	48	0.92	52	−4	6/10	.45
247	80	1.00	56	0	6/6	1.0

before that block was cut short; we report it in Appendix E and do not treat it as a completed cell.

Visibility as the dose. Question length varies across items (34 to 122 tokens here), so a single window already spreads visibility across items, and each item passes through several values of V as the window widens. Pooling the five treated cells and binning by per-item V: items with $V < 0.25$ lose 16.1 pp (n=118, $p<10^{-3}$) and items with V=1 lose 0.7 pp (n=135). Within items, a cluster-robust logistic regression of correctness on V across the ladder gives +0.68 (z=3.35, $p<.001$), and among the items whose correctness changes along the ladder, 21 are correct at high visibility and wrong at low against 2 in the opposite direction ($p<10^{-4}$). On treated rows, V is also the better one-parameter summary: AIC prefers $y \sim V$ (689.8) to $y \sim (w-c)$ (692.1) and to $y \sim (w-c) + |Q|$ (692.2).

The bins are not monotone throughout, and we state where. The $0.75 \le V < 1$ band loses 11.9 pp (n=59, p=.065); it holds nine distinct long-question items, six of them contributed by the single c+48 cell, four of which recover once V reaches 1. We therefore claim the window-level dose curve and the within-item trend, not a monotone per-item bin profile.

The consequence for any fix is that slack has to be counted in units of the question rather than in constant tokens. A slack the size of an English question leaves the median Telugu item at V=0.31, and Table 2 shows what that buys.

6 AutoWindow: one integer, and where it stops

The rule. Both quantities that Sections 4 and 5 identify are available before any forward pass. The trailing block c comes from the tokenizer and the chat template; a question-sized

Table 3: AutoWindow at a fixed eviction ratio of 0.75. Accuracy, with paired Δpp against each block's own uncompressed baseline in parentheses; n=100 per cell; star: McNemar $p < .05$. Panel (A) is Qwen3-4B on three of the eight languages measured; panel (B) is Qwen3-4B on English prompts carrying a JSON schema tail, where c is measured with the schema in place. c+16 was not rerun for panel (B).

	c	$\hat{w}$	baseline	w=64	c+16	AutoWindow
(A) languages						
en	25	43	93	93 (+0)	94 (+1)	95 (+2)
bn	107	183	73	57 (−16*)	66 (−7)	71 (−2)
te	167	247	56	37 (−19*)	50 (−6)	56 (+0)
(B) English with a JSON schema tail						
JSON-60	72	90	94	88 (−6)	—	96 (+2)
JSON-120	166	184	95	89 (−6*)	—	96 (+1)
JSON-200	213	231	94	89 (−5)	—	96 (+2)

slack can be estimated from a held-out sample of questions. We therefore set

$$\hat{w} \;=\; c + Q_{90},$$

one integer per (model, language, template), computed offline and without a delimiter. A per-item window w_i=$c + |Q_i|$ would give V_i=1 by construction, but it requires locating the question inside the prompt, which is exactly the assumption we are trying not to make (Corallo & Papotti, 2024). The language-level percentile is a deliberately weaker commitment: at $\hat{w}$ the window covers the entire question for 88 of 100 Bengali and 93 of 100 Telugu evaluation items, and the remaining decile is registered leftover rather than a claim.

The close. Panel (A) of Table 3 is the preregistered test, whose success criterion was $|\Delta| \leq 3$ pp against the uncompressed baseline. The formula returns +2, −2 and 0 pp for English, Bengali and Telugu, and gains +14 and +19 pp on Bengali and Telugu against the shipped default (16 fixed / 2 broken and 21/2; p=.001 and p<.001). The English-sized ablation c+16 behaves as Section 5 predicts: it recovers most of the hole and stops short of the baseline (−7 and −6 pp). Across the full eight-language grid, the languages whose trailing block already fits inside the default window stay within 3 pp of baseline throughout (Appendix D); the close is a statement about the languages the default window actually blinds.

Schema tails, and a construction we got wrong first. The same trailing-block logic applies when the tail is a response format rather than a translated instruction. An earlier run tested this and failed: it applied AutoWindow to English prompts carrying JSON schema tails but sized the window from the unpadded English instruction (w=41), which is smaller than the default it was meant to improve on, and at a 120-token schema that cost 23 pp. Measuring c with the schema in place gives 72, 166 and 213 tokens, hence $\hat{w}$ = 90, 184 and 231. Panel (B) shows the same formula then returning +2, +1 and +2 pp against baseline and +8, +7 and +7 pp against the default window, while breaking none of the items the default window answered correctly (8/0, 7/0 and 7/0). The 120-token schema is worth a second look: its trailing block, 166 tokens, is within one token of Telugu's 167. The same blindness, produced by a response format instead of a script, is removed by the same integer. Appendix F reports both runs side by side.

A second tokenizer. Gemma-3-4b-it tokenizes the same Bengali instruction to 26 tokens against Qwen's 102, and its Bengali questions to a median of 11 tokens, so both inputs to the rule move: c is 27/32/44 and Q_{90} is 18/18/20 for English, Bengali and Telugu, giving $\hat{w}$ = 45/50/64. Two things follow. For Telugu the rule returns exactly the constant that kvpress already ships, and Telugu is indeed flat there (−3 pp against its own baseline): a window rule that is worth running should also be able to say that the default is adequate. For Bengali, $\hat{w}$=50 leaves −6 pp, which misses the same ±3 pp criterion the 4B close met (Appendix G gives the full sweep on this tokenizer).

Scale. A Qwen3-8B slice on English and Bengali reproduces the phenomenon and not the close. The default window costs Bengali 17 pp ($p<.001$); $\hat{w}$ recovers +9 pp against it (11 fixed / 2 broken, $p=.022$) but remains 8 pp below baseline ($p=.008$). English is at ceiling throughout (96/97/96) and carries no information about either. We do not claim that the formula closes at 8B (Appendix H).

What the two residuals are not. The Gemma and 8B misses look like the same failure of the remedy, and both turn out not to be failures of visibility. At Gemma's $\hat{w}$=50 the median Bengali question occupies 11 tokens, so the window covers the question for nearly every item, and the same 5–6 pp gap is present at every window from 48 to 64 in the Gemma sweep — it is a plateau, not a cliff. At 8B, every one of the eight residual items has its whole question inside the window (their questions run 38 to 67 tokens, all below Q_{90}=76), seven of the eight echo the question verbatim in the output, and all twelve items whose question exceeds Q_{90} — the only items where blindness could still operate — are answered correctly under $\hat{w}$. What is left is a different kind of error: a dropped morpheme or a spelling variant inside an otherwise correct span, or an answer whose supporting sentence the evictor removed from the retained cache, with six of the seven unrecovered items sitting at the middle gold position. Widening the window turns off the blind mode; it does not, and should not be expected to, remove the ordinary cost of discarding three quarters of the cache.

Finally, the rule is free in the only budget that matters here. SnapKV keeps the observation window inside the retained cache, so a wider window does not buy more of it: every comparison in Table 3 is at the same eviction ratio, and AutoWindow costs one offline percentile per (model, language, template).

7 Analysis

Pooling the window sweep (3,000 generations over 500 items) and regressing correctness on a compression indicator and a blindness indicator $\mathbf{1}[w < c]$, with standard errors clustered by item, gives +0.09 log-odds for compression when the question is visible ($p=.21$) and −1.06 for blindness ($z=-9.4$, $p<.001$). Because uncompressed rows are never blind, the interaction term is identical to the blindness term; we report the operational form. Discarding three quarters of the cache is, on this task, statistically free as long as the scorer can see what was asked.

We also compared one-parameter descriptions of the compressed rows, and the comparison does not fully favour us. AIC prefers language fixed effects (2744) over the distance to the threshold ($w-c$) (3046), the binary blindness indicator (3075), and the raw window w (3220). The mechanism's prediction that ($w-c$) beats raw w holds. But on a five-language grid ($w-c$) is nearly collinear with language identity, and the sweep alone therefore cannot separate a threshold from a per-language constant. That separation comes from the experiments in which language is held fixed while c moves — the English pads (Section 4), the schema tails (Section 6) and the change of tokenizer — and from the dose ladder within a single language (Section 5). We state the AIC result rather than omit it because it is the one place where our headline sweep is genuinely ambiguous.

Results that do not support the account. A stuffed arithmetic-reasoning control does not reproduce the effect (Bengali −3 pp, ns): the failure needs a question whose content the scorer must match against passages, not merely a long prompt. A scorer that is designed never to look at the question (Devoto et al., 2025) damages Thai more than Bengali, which is a different failure from ours. Four-bit KV quantization damages languages in an order that is not the fertility order. Qwen3-4B and 8B share a tokenizer, so the scale slice is a replication rather than an independent sample of items. And a capacity mechanism does exist, but in a different regime: at aggressive eviction, where the retained cache approaches the length of the gold passage, even English items with long passages fail, whereas at the ratio used throughout this paper Bengali damage is flat across gold-length and retention strata. We do not use high-dose fertility slopes as evidence about the window.

8 Limitations

The failure needs a layout in which something long stands between the question and the end of the prompt. Instruction-last prompts and schema-after-content prompts are the biting cases; a chat deployment that puts the question last has $V{=}1$ by construction and nothing here applies to it. The production scope is likewise narrow: vLLM exposes no SnapKV-style evictor and the RocketKV path is opt-in, so we study a method family and one shipped opt-in constant rather than a default that most users are running today (Mao et al., 2026).

Two properties of the remedy are by design rather than by oversight. A language-level percentile leaves the longest decile of questions below $V{=}1$; and because SnapKV also protects the window it scores from, a very large w at a very aggressive eviction ratio would begin to tax the content budget. At the ratio used here that tax is small relative to the gold passage, which is why we did not need to separate scoring from protection to close the gap; at heavier ratios that separation may become necessary.

Our items are constructed rather than drawn from a standard long-context suite, and this is a deliberate trade. Published suites vary question length, instruction length, and language together, which is precisely the confound this paper has to hold fixed, but the price is that our absolute accuracies are not comparable to leaderboard numbers and that each cell rests on 100 items. Finally, one primary 4B model carries the headline result; the 8B slice shares its tokenizer, so it is a replication in scale rather than an independent sample, and a single contrast tokenizer cannot establish how the rule behaves across the space of tokenizers.

9 Conclusion

A token-denominated observation window is not language-neutral, because trailing instructions are not. Two quantities decide the outcome and both can be computed before any inference: c, the number of tokens standing between the question and the end of the prompt, and the slack the window leaves beyond it, which has to be counted in units of the question rather than in constants. Setting $w = c + Q_{90}$ from the tokenizer, the chat template and a held-out percentile removes the default-window hole on English, Bengali and Telugu at 4B and on English schema tails, returns the shipped default where the default is already adequate, and where it does not close leaves a residual that we can show is not a visibility failure. Fertility is the pathway rather than the ranking: the same failure is one instruction pad or one response schema away in English. The general lesson is cheaper than the specific one. Any constant that counts tokens inherits the tokenizer, and when such a constant gates what a model is allowed to look at, it should be derived from the tokenizer and the template instead of being shipped as a number.

Ethics statement

This work uses publicly released question-answering corpora and publicly released model weights; it involves no human subjects and no personal data. The failure it documents falls unevenly on users who write in scripts that current tokenizers encode inefficiently, so we have tried to describe it in terms of the mechanism rather than in terms of language rankings, and to avoid comparisons of absolute accuracy between languages that our item construction does not support. The remedy we propose is cheap and requires no additional data collection.

AI use statement

The experimental harness, the literature survey, and a first draft of this paper were produced with substantial assistance from large language models (Grok 4.6 and Claude). Every number reported here was recomputed from the raw generation records by a second model working from the stored outputs rather than from the first draft's tables, and the discrepancies that pass found — including two incorrect counts in an internal analysis note and several incorrect bibliography entries — were corrected before submission. The predictions

for each experiment were written and committed before the corresponding generations were run. The authors take full responsibility for all claims in this paper.

Reproducibility statement

Configurations, seeds, prompts and the scoring rule are in the supplementary code. The two constants are produced by scripts, not copied between documents: measure_c.py (and measure_c_schema.py for schema tails) computes c from the run tokenizer and chat template, and measure_q.py computes Q_{90} from a held-out question split. The dose-response readout is produced by v_trace_bins.py, which was committed before the corresponding data existed. Nine generation stores back the tables: cliff_multi, cliff_en, autowin, autowin_q90, autowin_8b, cliff_gemma, schema_fix, gemma_q90 and v_trace. Each table cell is recomputed from the raw generations in those stores, and comparisons never cross a stack boundary (Appendix J).

References

Mikel Artetxe, Sebastian Ruder, and Dani Yogatama. On the cross-lingual transferability of monolingual representations. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pp. 4623–4637, 2020. doi: 10.18653/v1/2020.acl-main.421.

Alex Chen, Renato Geh, Aditya Grover, Guy Van Den Broeck, and Daniel Mingyi Israel. The pitfalls of KV cache compression. In Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 41530–41553, 2026. doi: 10.18653/v1/2026.acl-long.1926. ACL Anthology 2026.acl-long.1926.

Jonathan H. Clark, Eunsol Choi, Michael Collins, Dan Garrette, Tom Kwiatkowski, Vitaly Nikolaev, and Jennimaria Palomaki. TyDi QA: A benchmark for information-seeking question answering in typologically diverse languages. Transactions of the Association for Computational Linguistics, 8:454–470, 2020. doi: 10.1162/tacl_a_00317.

Giulio Corallo and Paolo Papotti. FINCH: Prompt-guided key-value cache compression for large language models. Transactions of the Association for Computational Linguistics, 12:1517–1532, 2024. doi: 10.1162/tacl_a_00716. Anthology 2024.tacl-1.83.

Alessio Devoto, Maximilian Jeblick, and Simon Jégou. Expected attention: KV cache compression by estimating attention from future queries distribution. arXiv preprint arXiv:2510.00636, 2025.

Gabriel Garcia. Protection is (nearly) all you need: Structural protection dominates scoring in globally capped KV eviction. arXiv preprint arXiv:2605.18053, 2026.

Gemma Team, Google DeepMind. Gemma 3 technical report, 2025. arXiv:2503.19786.

Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In Advances in Neural Information Processing Systems, 2024.

Daming Luo, Christy Liang, and Junyu Xuan. How query visibility changes KV-cache compression rankings: A matched-budget audit. arXiv preprint arXiv:2607.11942, 2026.

Weian Mao, Yukang Chen, Wei Huang, Shuai Yang, Luozhou Wang, and Song Han. KV cache compression and its infra problems. NVIDIA Efficient AI Lab research blog, 12 June 2026, 2026. Cited against a serving-default claim.

Kelly Marchisio, Saurabh Dash, Hongyu Chen, Dennis Aumiller, Ahmet Üstün, Sara Hooker, and Sebastian Ruder. How does quantization affect multilingual LLMs? arXiv preprint arXiv:2407.03211, 2024.

Benjamin Marie and Atsushi Fujita. The uneven impact of post-training quantization in machine translation. arXiv preprint arXiv:2508.20893, 2025.

Piotr Nawrot, Jianing Li, Renjie Huang, Sebastian Ruder, Kelly Marchisio, and Edoardo Ponti. The sparse frontier: Sparse attention trade-offs in transformer LLMs. In Findings of the Association for Computational Linguistics: ACL 2026, pp. 38667–38701, 2026. doi: 10.18653/v1/2026.findings-acl.1926. ACL Anthology 2026.findings-acl.1926 (Findings volume; the shared number with Chen et al. 2026.acl-long.1926 is a coincidence across tracks, verified against the Anthology record); arXiv:2504.17768, whose listing names the second author Robert Li.

Aleksandar Petrov, Emanuele La Malfa, Philip Torr, and Adel Bibi. Language model tokenizers introduce unfairness between languages. In Advances in Neural Information Processing Systems, 2023.

Qwen Team. Qwen3 technical report, 2025. arXiv:2505.09388.

Liang Zhao, Xiaocheng Feng, Weihong Zhong, Lei Huang, Kun Zhu, Baoxin Wang, Dayong Wu, Guoping Hu, Ting Liu, and Bing Qin. Question tells you where the answer is: Intention-aware long-context KV cache compression. In Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 27153–27169, 2026. doi: 10.18653/v1/2026.acl-long.1250. ACL Anthology 2026.acl-long.1250.

A How last-w scoring works

SnapKV forms $q_{\text{obs}} = q[:, :, -w:]$, scores the prefix keys against that slice, keeps the highest-scoring entries under the compression ratio, and protects the window itself from eviction. The kvpress implementation defaults to w=64 with a pooling kernel of size 5. The TensorRT-LLM RocketKV path defaults to w=32 and skips sparse KV entirely when the sequence is shorter than its prompt budget. Because the window is retained rather than added, raising w at a fixed compression ratio does not increase the amount of cache that survives: it changes which entries are selected, not how many.

B Measuring c and Q_{90}

c is produced by measure_c.py. A probe prompt is wrapped in the model's chat template with add_generation_prompt; if the probe question's token ids appear as a contiguous subsequence, c is the number of tokens after that span (after_question); otherwise $c = I + s$, where s is the assistant-header suffix measured on a minimal turn (fallback_I_plus_suffix). Qwen3-4B takes the fallback path with s=5 and Gemma-3-4b-it the direct path with s=6. We verified the fallback against a direct offset-mapping measurement on real prompts: for all eight Qwen languages the two agree within one token, the exception being a single token of Thai where the boundary falls inside a merged token.

For a padded or schema-carrying instruction, measure_c_schema.py builds the instruction with the same padding routine the item builder uses and reports c for the padded string, so the measured constant and the generated prompt cannot drift apart.

Q_{90} is produced by measure_q.py as the 90th percentile of len(tokenizer.encode(question)) on a split that excludes the 100 evaluation items: XQuAD validation after index 100, and TyDiQA-GoldP train with every evaluation question id removed. The evaluation items themselves are never used to estimate it.

C The scoring rule and its robustness

An answer counts as correct if, after NFKC normalization, lowercasing, punctuation stripping and language-aware tokenization (character-level for scripts written without word separators; article stripping only for languages that have articles), some gold span appears inside the marked answer span or inside the first sentence of the prose that precedes the marker. Whole-output containment would over-credit answers that merely quote a passage;

marker-only scoring under-credits models that state the answer in prose and then emit a reformatted token after the marker, and that failure rate differs by language, which would manufacture exactly the kind of per-language gap this paper is about. An earlier frozen rule recorded during generation was marker-only; we never quote it and recompute every cell offline.

The choice does not carry the results. Under a stricter marker-only rule the Telugu dose ladder still shows the same shape: the uncompressed baseline scores 50% rather than 56%, c+4 scores 35% and $c{+}Q_{90}$ scores 47%, so the hole is -15 pp instead of -18 pp and the close is -3 pp instead of 0 pp. Table 4 repeats the 8B Bengali cells under two further scorers.

Table 4: Robustness of the 8B Bengali cells to the scoring rule. The spelling fold maps two pairs of Bengali graphemes that the model interchanges; gold-token recall accepts a prediction covering at least 70% of the gold tokens. The ranking and the sign of every comparison are unchanged.

scorer	baseline	w=64	$\hat{w}$	$\hat{w}-$ baseline
containment (used throughout)	81	64	73	−8
containment after a spelling fold	81	65	75	−6
gold-token recall ≥ 0.7	85	70	79	−6

D Full window, pad, and c+16 grids

Table 5 gives the measured constants for all eight languages. Table 6 gives the eight-language run of the c+16 ablation: the five languages whose trailing block already fits inside the default window stay within 3 pp of baseline at both settings, with a marginal Vietnamese exception at the default window; only Bengali and Telugu carry a hole for c+16 to close, and it does not close it. Table 7 gives the English pad sweep plotted in Figure 2b.

Table 5: Measured constants, Qwen3-4B. I is the instruction length, $c{=}I{+}s$ with s=5, and Q_{90} is the held-out question-length percentile (n=1090/2387/5563 for en/bn/te; Q_{50} is 12/46/54 and $Q_{\max}$ is 36/141/170). Q_{90} was measured for the three languages the AutoWindow arms use.

	en	zh	es	vi	th	sw	bn	te
I	20	24	30	34	40	42	102	162
c	25	29	35	39	45	47	107	167
c+16	41	45	51	55	61	63	123	183
Q_{90}	18	—	—	—	—	—	76	80
$\hat{w} = c{+}Q_{90}$	43	—	—	—	—	—	183	247

Table 6: The c+16 ablation across eight languages. Accuracy, with paired Δpp against the uncompressed baseline in parentheses; n=100; star: McNemar $p < .05$.

	c	c+16	baseline	w=64	c+16
en	25	41	93	93 (+0)	94 (+1)
zh	29	45	83	83 (+0)	85 (+2)
es	35	51	88	86 (−2)	88 (+0)
vi	39	55	83	78 (−5)	81 (−2)
th	45	61	85	85 (+0)	84 (−1)
sw	47	63	56	53 (−3)	56 (+0)
bn	107	123	73	57 (−16*)	66 (−7)
te	167	183	56	37 (−19*)	50 (−6)

Table 7: English pad sweep. Accuracy, n=100 per cell, eviction ratio 0.75. This store carries no uncompressed baseline, so the comparison is across windows; the bold cell in each row is the first window exceeding that pad's c, and it is also that row's maximum.

pad	c	w=32	56	80	104	144
48	57	81	87	92	92	92
64	73	86	90	94	93	93
96	105	82	90	89	90	93
128	137	78	85	89	89	91

E The dose ladder in full

The Telugu ladder is Table 2 in the main text. Binning the five treated cells by per-item visibility gives: $V < 0.25$, n=118, −16.1 pp (6 fixed / 25 broken, p=9×10^{-4}); $0.25 \le V < 0.5$, n=104, −4.8 pp; $0.5 \le V < 0.75$, n=84, −2.4 pp; $0.75 \le V < 1$, n=59, −11.9 pp (2/9, p=.065); V=1, n=135, +0.7 pp (9/8).

The $0.75 \le V < 1$ band is the one that breaks monotonicity, so we describe its contents. It contains nine distinct items with questions of 40 to 81 tokens; six of the nine enter through the c+48 cell alone, and four of the nine are answered correctly once the window reaches V=1. The band is not significant on its own (p=.065) and it represents 59 of the 500 treated observations. Read together with the V=1 bin, it says that visibility close to 1 is not yet equivalent to visibility at 1 for long questions — which is the same statement as the registered leftover of a language-level percentile, seen per item.

The partial Bengali block reproduces the first two rungs of the ladder before it was cut short: at c+4 (w=111) accuracy falls from 73 to 56 (−17 pp, 3 fixed / 20 broken, p=.0005), and at c+16 (w=123) it is 66 (−7 pp). A third cell at c+32 completed only 43 of 100 items and is reported here for completeness (−2.3 pp on the items that finished) but is used nowhere in the paper. The within-item sign test on the Bengali block gives 13 items correct at high visibility against 1 in the opposite direction (p=.002); its AIC comparison ties between V and $(w-c)$ at 318.4, which we report as a tie rather than as support.

F Schema tails: the mis-sized run and the rerun

The first schema run applied the AutoWindow rule but computed c from the unpadded English instruction, giving w=41 — smaller than the default w=64 it was supposed to improve on. It therefore tested a window below the threshold rather than the rule, and the result was correspondingly bad: against uncompressed baselines of 94/95/94 for the 60-, 120- and 200-token schema tails, the default window scored 88/89/89 (−6, −6*, −5 pp) and the mis-sized window scored 88/72/75 (−6, **−23***, **−19*** pp). The mis-sizing is itself an instance of the paper's claim: an English-derived constant applied to a prompt whose trailing block is much longer.

The rerun measures c with the schema in place (72/166/213) and is reported in panel (B) of Table 3. The two runs share their uncompressed and default-window cells, which the rerun reproduced exactly, generation for generation.

G A second tokenizer: Gemma-3-4b-it

Gemma-3-4b-it encodes the same instructions much more compactly than Qwen3-4B: c is 27/32/44 for English, Bengali and Telugu against Qwen's 25/107/167, and its questions are shorter in tokens as well (Q_{90} = 18/18/20 against 18/76/80). The threshold should therefore move down with the tokenizer, and the window sweep in Table 8 is consistent with that: Bengali is damaged at the two windows below its c and drifts to a non-significant −5 pp at the two above it, while Telugu — whose Qwen counterpart loses 19 points at w=64 — shows no comparable hole anywhere on the grid. English is flat to slightly positive throughout.

Table 8: Gemma-3-4b-it window sweep. Paired Δpp against the Gemma baseline (accuracy 75/62/37 for en/bn/te), n=100, star: McNemar $p < .05$. Not pooled with any Qwen cell.

	c	w=16	24	32	48	64
en	27	+2	+1	+3	+4	+3
bn	32	-13^*	-10^*	-9^*	−5	−5
te	44	−3	−4	−3	−1	−3

Two qualifications belong with this table. Bengali at w=32, which is c exactly, is still −9 pp, so the step is not a clean discontinuity at c here either; and Telugu's baseline of 37% is low enough that the sweep would not resolve a small hole. The AutoWindow arm on this tokenizer gives $\hat{w}$ = 45/50/64 and returns +3/−6/−3 pp against baseline, against +3/−5/−3 at the default window. For Telugu the rule selects the default constant itself. For Bengali it does not close, and the main text explains why we read that residual as a plateau rather than a threshold effect: it is present at every window from 48 upward, and at $\hat{w}$=50 the median Bengali question of 11 tokens is entirely inside the window.

H Scale: Qwen3-8B

The scale slice covers English and Bengali with the same formula and the same 100 items per cell. English is at ceiling and carries no information: 96 baseline, 97 at the default window, 96 at $\hat{w}$=43. Bengali reproduces the phenomenon: 81 baseline, 64 at the default window (−17 pp, 1 fixed / 18 broken, p<.001), and 73 at $\hat{w}$=183, which recovers +9 pp against the default window (11/2, p=.022) but stays 8 pp below baseline (0/8, p=.008). The registered criterion was ±3 pp, so this is a miss, and we report it as one.

What the residual is not is worth stating precisely, because it is the same diagnosis that applies to Gemma. Of the eight items that the baseline answers and $\hat{w}$ does not, all eight have questions shorter than Q_{90} (38 to 67 tokens), so the window covers the question entirely for every one of them; seven of the eight reproduce the question verbatim in the generated text; and all twelve items whose question exceeds Q_{90} — the only items where the language-level percentile could still leave the scorer blind — are answered correctly under $\hat{w}$. Among the items that the default window breaks and $\hat{w}$ does not recover, three of seven run to the decode cap without ever emitting an answer marker, against none of the eleven items it does recover, and six of the seven sit at the middle gold position. The errors that remain are near-copies of the gold span (a dropped morpheme, an interchanged grapheme) or answers whose supporting sentence did not survive eviction — ordinary compression damage, not blindness.

I Frozen instructions and prompt layout

The QA instructions are one line per language, frozen under a prompt version key so that a change to them would invalidate the stored runs rather than silently alter them. They carry the same communicative content — answer from the passages above, end with a marked span — and differ only in how many tokens the model's tokenizer needs for it: 20 for English, 102 for Bengali, 162 for Telugu on Qwen3-4B. Every instruction-last item has the layout

```
[gold + distractor passages, packed to 8k tokens]
<question>
<instruction>
<chat suffix>          # Qwen: <|im_end|>\n<|im_start|>assistant\n
```

and the observation window is the last w tokens of that concatenation. In English, for instance:

```
... Paris is the capital of France. ...
What is the capital of France?
```

Answer the question using only the passages above.
End your reply with '#### <exact answer span>'.
<|im_end|>
<|im_start|>assistant

The padded conditions prepend content-free filler to the instruction so that the marked-span sentence stays adjacent to generation and only the token count changes; the schema conditions prepend a JSON response specification in the same position.

J Measurement transcripts and run provenance

Stacks. Each generation record stores a stack descriptor (GPU, driver, CUDA, torch, transformers, kvpress) and every comparison in this paper is within one descriptor. The Qwen3-4B cells share one stack, the Gemma cells another, and the Qwen3-8B slice a third; we never pool across them.

Determinism, measured. The follow-up runs were executed weeks after the original ones, on new machines, and they re-ran cells that the earlier stores already contained: the uncompressed and default-window cells of the schema conditions, the uncompressed and default-window cells of the Gemma languages, and three Telugu cells shared with the AutoWindow stores. Across those 1,200 shared generations, the re-run text is byte-identical to the original, which is what allows us to treat the re-measured constants and the earlier tables as describing the same system.

Decode cap. Every arm reported in the main text uses a 384-token decode cap. Two supporting results quoted from earlier work were measured under a 128-token cap — the instruction-first layout swap and the window-widening comparison in Appendix K — and we flag the difference here rather than leave it to be discovered in a log. The cap matters: at 128 tokens, answers in high-fertility scripts are truncated at a rate that varies by language, which inflates apparent per-language damage. That is itself an instance of this paper's subject, and it is why the main arms were re-run at 384.

K Two further levers on visibility

Two earlier interventions move the same quantity and are consistent with the account in the main text; both were measured under the 128-token decode cap noted above, so we report them as supporting rather than as headline evidence. Moving the instruction to the front of the prompt, which places the question inside every window without changing the press or the retained budget, recovers Bengali by 14 pp at a fixed ratio (34 fixed / 6 broken, $p<10^{-5}$) and by 22 pp under an absolute cache budget (51/7, $p<10^{-8}$); English is unchanged and Thai loses 6.5 pp, so this is a lever on the mechanism rather than a deployment recommendation. Widening the window from 64 to 128 tokens at a fixed ratio recovers Bengali by 10.5 pp (22 fixed / 1 broken, $p<10^{-5}$) and saturates there: going on to 256 tokens changes nothing (-0.5 pp, 8/9). Saturation at a window well below $c+Q_{90}$ for that setting is consistent with the dose curve in Section 5.

L Production scope

vLLM exposes KV-cache dtype and scheduler token budgets, not a SnapKV-style evictor. TensorRT-LLM implements last-w scoring as an opt-in sparse-attention backend whose default window is 32 tokens, and skips it entirely below a prompt-length threshold. Mao et al. (2026) argue that research evictors rarely ship as production defaults, and we cite that against ourselves: the constant we study is a research default and one opt-in production constant, not something most deployments are running today. The reason to fix it now is that the fix is a measurement rather than a redesign.